LAB 04 Input forms, validation and local UI state

Unit 2 • 3-hour guided laboratory • CLO-5

Learning goal. Build interactive forms using TextFormField, selection widgets, a Form validator, controllers and setState.

Core concepts. FormState, GlobalKey, TextEditingController disposal, CheckboxListTile, RadioGroup alternatives, SwitchListTile, dialogs and SnackBar feedback.

Before you start. Lab 03 UI knowledge; use an independent new project or replace lib/main.dart between tasks.

PROJECT / PACKAGE SETUP — run in Terminal

```
flutter create --platforms=android mad_lab04
cd mad_lab04
```

Solved task 1 | Validated lab registration form

Problem and objective. A student must supply name, registration number, select a lab section and accept an academic-integrity declaration before submission.

lib/main.dart

```
import 'package:flutter/material.dart';

void main() => runApp(const MaterialApp(home: RegisterPage()));

class RegisterPage extends StatefulWidget {
  const RegisterPage({super.key});
  @override
  State<RegisterPage> createState() => _RegisterPageState();
}

class _RegisterPageState extends State<RegisterPage> {
  final key = GlobalKey<FormState>();
  final name = TextEditingController();
  final roll = TextEditingController();
  String section = 'A';
  bool agreed = false;

  @override
  void dispose() {
    name.dispose(); roll.dispose(); super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Register for MAD lab')),
      body: Form(key: key, child: ListView(
        padding: const EdgeInsets.all(20), children: [
          TextFormField(
            controller: name,
            decoration: const InputDecoration(labelText: 'Full name'),
            validator: (s) => (s == null || s.trim().length < 3)
                ? 'Enter at least 3 characters' : null,
          ),
          TextFormField(
            controller: roll,
            decoration: const InputDecoration(labelText: 'Roll number'),
            validator: (s) => (s == null || s.trim().isEmpty)
                ? 'Roll number is required' : null,
          ),
          DropdownButtonFormField<String>(
            initialValue: section,
            items: const [
              DropdownMenuItem(value: 'A', child: Text('Section A')),
              DropdownMenuItem(value: 'B', child: Text('Section B')),
            ],
            onChanged: (v) => setState(() => section = v ?? 'A'),
```

```
          ),
          CheckboxListTile(
            value: agreed,
            title: const Text('I will submit original work'),
            onChanged: (v) => setState(() => agreed = v ?? false),
          ),
          FilledButton(
            onPressed: () {
              if (!key.currentState!.validate()) return;
              if (!agreed) {
                ScaffoldMessenger.of(context).showSnackBar(const SnackBar(
                  content: Text('Accept the declaration first')));
                return;
              }
              showDialog<void>(context: context, builder: (context) =>
                AlertDialog(
                  title: const Text('Registered'),
                  content: Text('${name.text.trim()} (${roll.text.trim()}) '
                      'in Section $section'),
                  actions: [TextButton(onPressed: () => Navigator.pop(context),
                      child: const Text('OK'))],
                ));
            },
            child: const Text('Register'),
          ),
        ],
      )),
    );
  }
}
```

How the solution works. Validators return a message for invalid values and null for valid values. GlobalKey<FormState> invokes all attached field validators. The declaration is additional state checked after the Form is valid. Controllers are disposed to release listeners.

Expected output. Invalid submissions show field-specific messages; a complete registration shows a summary dialog.

Check your work. Test blank fields, two-character names, unchecked declaration and a valid submission; demonstrate each validation result.

Solved task 2 | Radio-like choice, switch and computed bill

Problem and objective. Create a meal selector using SegmentedButton as an accessible mutually-exclusive choice, a delivery switch and live calculated total.

lib/main.dart

```
import 'package:flutter/material.dart';

void main() => runApp(const MaterialApp(home: MealPage()));

class MealPage extends StatefulWidget {
  const MealPage({super.key});
  @override
  State<MealPage> createState() => _MealPageState();
}

class _MealPageState extends State<MealPage> {
  String meal = 'Lunch';
  int quantity = 1;
  bool delivery = false;
  final prices = {'Breakfast': 200, 'Lunch': 350, 'Dinner': 450};

  @override
  Widget build(BuildContext context) {
    final total = prices[meal]! * quantity + (delivery ? 100 : 0);
    return Scaffold(
      appBar: AppBar(title: const Text('Meal estimator')),
      body: ListView(padding: const EdgeInsets.all(16), children: [
        SegmentedButton<String>(
          segments: const [
```

```
            ButtonSegment(value: 'Breakfast', label: Text('Breakfast')),
            ButtonSegment(value: 'Lunch', label: Text('Lunch')),
            ButtonSegment(value: 'Dinner', label: Text('Dinner')),
          ],
          selected: {meal},
          onSelectionChanged: (v) => setState(() => meal = v.first),
        ),
        Row(children: [
          const Expanded(child: Text('Quantity')),
          IconButton(onPressed: quantity > 1
              ? () => setState(() => quantity--) : null,
              icon: const Icon(Icons.remove)),
          Text('$quantity'),
          IconButton(onPressed: () => setState(() => quantity++),
              icon: const Icon(Icons.add)),
        ]),
        SwitchListTile(
          title: const Text('Home delivery (Rs. 100)'),
          value: delivery,
          onChanged: (v) => setState(() => delivery = v),
        ),
        const Divider(),
        Text('Total: Rs. $total',
            style: Theme.of(context).textTheme.headlineMedium),
      ]),
    );
  }
}
```

How the solution works. The meal selection acts like a group of radio buttons: exactly one selection is active. The total is derived in build from the current state, so it cannot go stale. The disabled decrement button enforces a minimum quantity of one.

Expected output. Meal choice, quantity buttons and delivery switch immediately modify the estimated total.

Check your work. Verify the quantity never falls below 1 and derive the expected price for Dinner × 2 + delivery.

Scenario-based practice — unsolved

Scenario 1. Course feedback. Design a feedback form requiring a rating, bounded comment length and anonymous opt-in. Specify validation, accessible controls and safe reset behavior.

Scenario 2. Admission fee. A student selects program, transport and hostel needs; compute fees but ensure no negative quantity and display clear error messages.

Technical viva — justify your reasoning

1. Why does calling setState outside State produce an architectural error?
2. Why dispose TextEditingController?
3. Why is a validator returning null interpreted as successful validation?
4. What is the danger of storing a computed total in a separate mutable field?
5. When is SnackBar more appropriate than AlertDialog?
6. Which state should survive an app restart, and would setState alone achieve that?

Submission and self-check

Submit valid and invalid screenshots, the exact input test cases and a Git commit labeled feat: validated lab form.